Desenvolvimento Web com Spring Boot e React no VS Code

1 Usando VSCode para criar App WEB Springboot com React

Vamos criar e executar uma aplicação web com **Spring Boot no backend** e **React no frontend**, usando o Visual Studio Code (VS Code). Durante o desenvolvimento, os dois componentes serão executados separadamente. O Nginx não é necessário nesta etapa.

2 Preparar o ambiente

1. Instale um JDK LTS, por exemplo, Java 21. (geralmente já vai estar instalado)
2. Instale uma versão LTS do Node.js, que inclui o npm.
3. No VS Code, abra `Ctrl+Shift+X` e instale as extensões *Extension Pack for Java* e *Spring Boot Extension Pack*.
4. Abra um terminal no VS Code e confira as instalações:

```
java -version
node -v
npm -v
```

3 Criar o backend Spring Boot

1. Acesse start.spring.io.
2. Selecione *Maven*, *Java* e uma versão estável do Spring Boot compatível com o JDK instalado. Use, por exemplo, `Group: com.exemplo`, `Artifact: backend`, `Packaging: Jar` e `Java: 21`.
3. Adicione a dependência *Spring Web* e gere o projeto.
4. Extraia o arquivo gerado para uma pasta chamada `backend`, dentro da pasta principal do projeto.
5. No VS Code, abra a pasta `backend`. Aguarde a importação do Maven.
6. No terminal integrado, execute o comando correspondente ao sistema:

```
# Windows (PowerShell)
.\mvnw.cmd spring-boot:run

# Linux
./mvnw spring-boot:run
```

O backend normalmente ficará disponível em `http://localhost:8080`. Mantenha esse terminal aberto enquanto desenvolve.

4 Criar o frontend React

Abra outro terminal no VS Code. A partir da pasta principal do projeto (a pasta que contém `backend`), execute:

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev
```

O Vite exibirá um endereço, normalmente http://localhost:5173. Abra-o no navegador. Mantenha também esse terminal aberto.

5 Conectar React ao Spring Boot

Para evitar configurar CORS durante o desenvolvimento, configure o proxy do Vite. No arquivo `frontend/vite.config.js`, use:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': 'http://localhost:8080'
    }
  }
})
```

Salve o arquivo. Se o Vite não reiniciar automaticamente, pare-o com `Ctrl+C` e execute novamente `npm run dev`. Chamadas do frontend para caminhos iniciados por `/api` serão encaminhadas ao Spring Boot.

6 Exemplo completo: listar mercadorias

6.1 1. Criar o endpoint no backend

No projeto Spring Boot, localize o pacote da classe principal. Crie nesse mesmo pacote o arquivo `MercadoriaController.java`. O exemplo abaixo usa `com.exemplo.backend`; se o projeto gerado tiver outro pacote, ajuste a primeira linha para corresponder ao pacote real.

```
package com.exemplo.backend;

import java.util.List;
import java.util.Map;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/mercadorias")
public class MercadoriaController {

    @GetMapping
    public List<Map<String, Object>> listar() {
        return List.of(
```

```
            Map.of("id", 1, "nome", "Teclado", "preco", 120.00),
            Map.of("id", 2, "nome", "Mouse", "preco", 65.00)
        );
    }
}
```

Com o Spring Boot em execução, abra http://localhost:8080/api/mercadorias. A resposta deverá ser uma lista JSON com duas mercadorias. Este exemplo usa dados fixos; ainda não acessa banco de dados.

6.2 2. Exibir os dados no React

Substitua o conteúdo de `frontend/src/App.jsx` por:

```
import { useEffect, useState } from 'react'

function App() {
  const [mercadorias, setMercadorias] = useState([])
  const [erro, setErro] = useState('')

  useEffect(() => {
    fetch('/api/mercadorias')
      .then(resposta => {
        if (!resposta.ok) {
          throw new Error('Falha na consulta')
        }
        return resposta.json()
      })
      .then(dados => setMercadorias(dados))
      .catch(() => setErro('Nao foi possivel carregar as mercadorias'))
  }, [])

  return (
    <main>
      <h1>Estoque de mercadorias</h1>
      {erro && <p>{erro}</p>}

      <table>
        <thead>
          <tr>
            <th>ID</th>
            <th>Nome</th>
            <th>Preco</th>
          </tr>
        </thead>
        <tbody>
          {mercadorias.map(item => (
            <tr key={item.id}>
              <td>{item.id}</td>
              <td>{item.nome}</td>
              <td>
                {item.preco.toLocaleString('pt-BR', {
                  style: 'currency',
                  currency: 'BRL'
                })}
              </td>
            </tr>
          ))}
        </tbody>
```

```
      </table>
    </main>
  )
}

export default App
```

7 Executar e testar

Mantenha dois terminais abertos:

- **Terminal 1 – backend:** na pasta `backend`, execute `.\mvnw.cmd spring-boot:run` no Windows ou `./mvnw spring-boot:run` no Linux.
- **Terminal 2 – frontend:** na pasta `frontend`, execute `npm run dev`.

Acesse http://localhost:5173. A tabela deverá mostrar Teclado e Mouse. Se houver erro, confirme que ambos os processos estão em execução, que o endpoint responde em http://localhost:8080/api/mercadorias e que o proxy foi salvo em `vite.config.js`.

8 Estrutura final

```
projeto/
|-- backend/
|   |-- pom.xml
|   |-- mvnw
|   `-- src/main/java/com/exemplo/backend/
|       |-- BackendApplication.java
|       `-- MercadoriaController.java
`-- frontend/
    |-- package.json
    |-- vite.config.js
    `-- src/
        |-- App.jsx
        `-- main.jsx
```

Observação: o exemplo usa dados em memória apenas para demonstrar a comunicação entre React e Spring Boot. Banco de dados, autenticação, autorização e configuração de produção devem ser adicionados conforme os requisitos da aplicação. O Nginx pode ser configurado posteriormente para servir os arquivos de produção do React e encaminhar as requisições `/api` ao backend.